

NLP Risk Engine + Index Rebalancer + Stress Tester

Turning news & social chatter into structured risk signals

Candidate: Mehwish | College: VIT | Solo submission

Problem

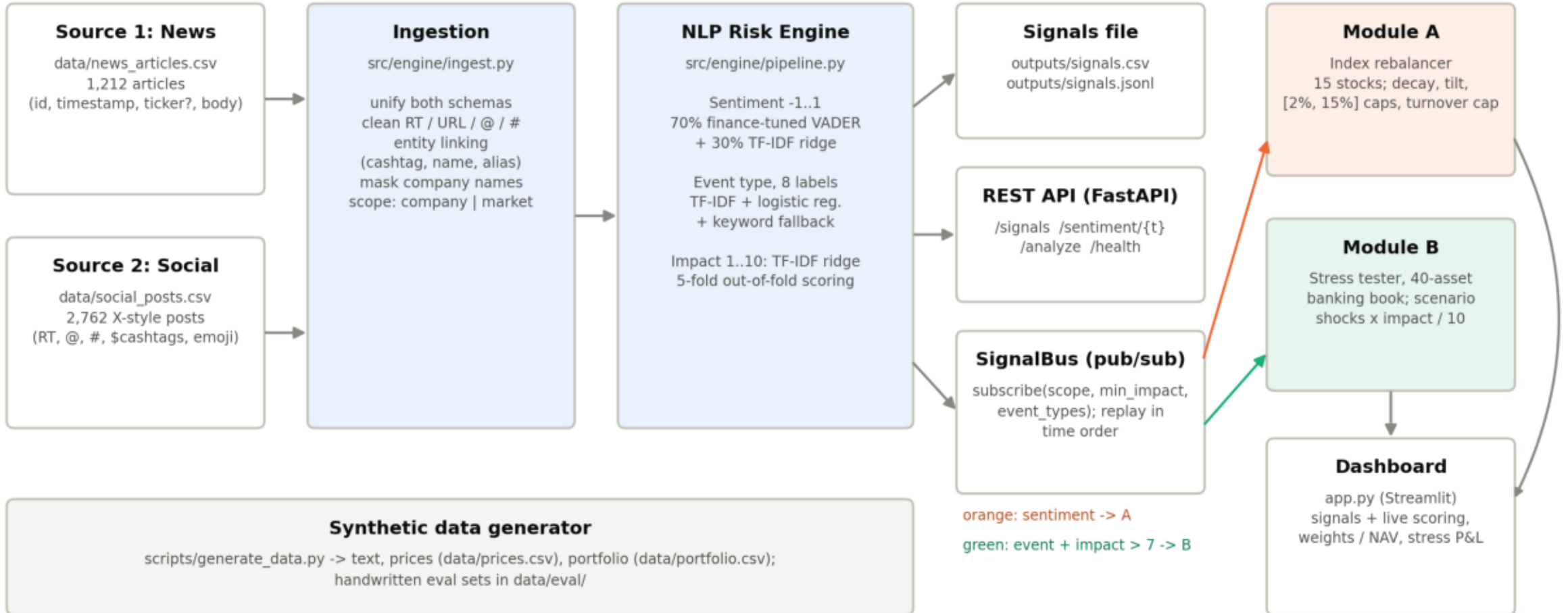
- Market-moving information arrives as unstructured text (news, X posts) faster than analysts can read it.
- Risk and portfolio teams need it as numbers they can act on: how positive/negative, what kind of event, how severe.

Approach

- One NLP Risk Engine: 2 sources in, one signal per text out: sentiment (-1..1), event type, impact (1..10).
- Signals published via file, REST API and an in-process pub/sub bus.
- Both downstream modules built on the bus:
 - A: sentiment-tilted index of 15 S&P 100 stocks
 - B: event-triggered stress test of a 40-asset wholesale banking book
- One Streamlit dashboard for the live demo.

System design and data flow

NLP Risk Engine: architecture and data flow



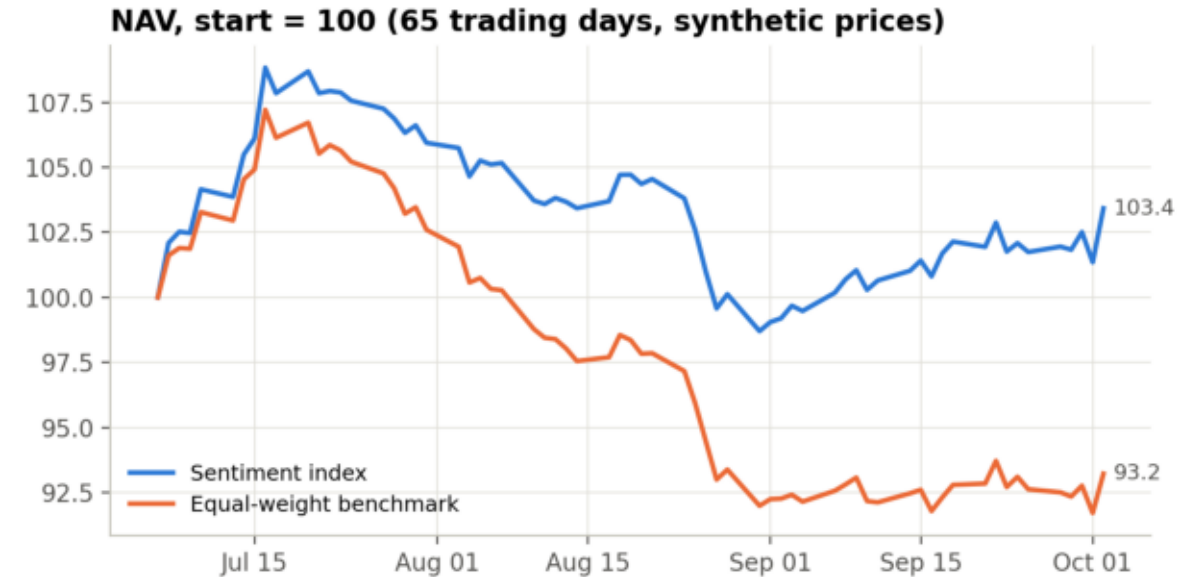
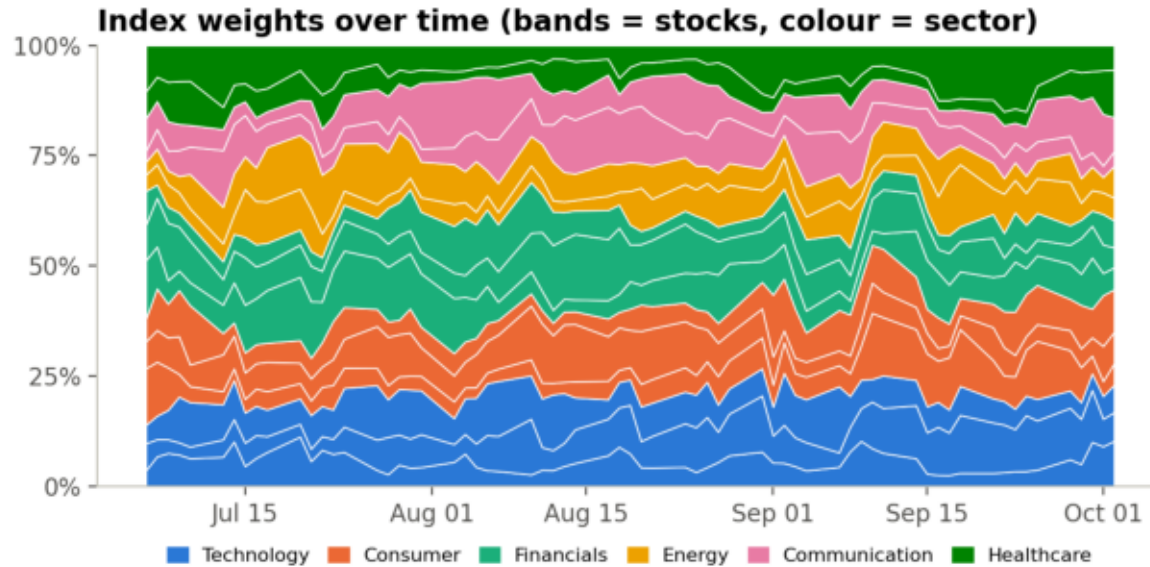
Implementation: the NLP Risk Engine

- Ingestion: 1,212 news + 2,762 social texts; social cleaned (RT, URLs, @, #); entity linking by cashtag/name/alias -> 3,547 company + 427 market-wide signals.
- Company names masked to 'company' so models learn language, not 'AAPL is usually positive'.
- Sentiment: finance-tuned VADER lexicon blended with TF-IDF ridge. Event: TF-IDF + logistic regression. Impact: regressor on text + source type.
- Every signal is scored out-of-fold (5-fold), so modules never consume memorised training rows.
- Tech: Python 3.13, pandas, scikit-learn, vaderSentiment, FastAPI, Streamlit, Plotly; 27 pytest tests.

Metric	Synthetic (OOF)	Holdout (20)
Sentiment Pearson r	0.908	0.878
Sentiment 3-class acc	0.899	0.750
Event accuracy	0.969	0.900
Impact MAE (1-10)	0.85	1.18
Lexicon-only 3-class acc	-	0.800

Honest read: synthetic scores are high because test text shares the generator's templates. The 20 handwritten holdout texts (never tuned on) are the real generalisation check: event 0.90, sentiment r 0.88. On them the plain lexicon (3-class 0.80) beats the blend (0.75): the ML part overfits template wording.

Results, Module A: sentiment-driven index rebalancing



+3.4%

index total return (EW -6.8%)

1.06

Sharpe (EW -2.08)

-9.3%

max drawdown (EW -14.5%)

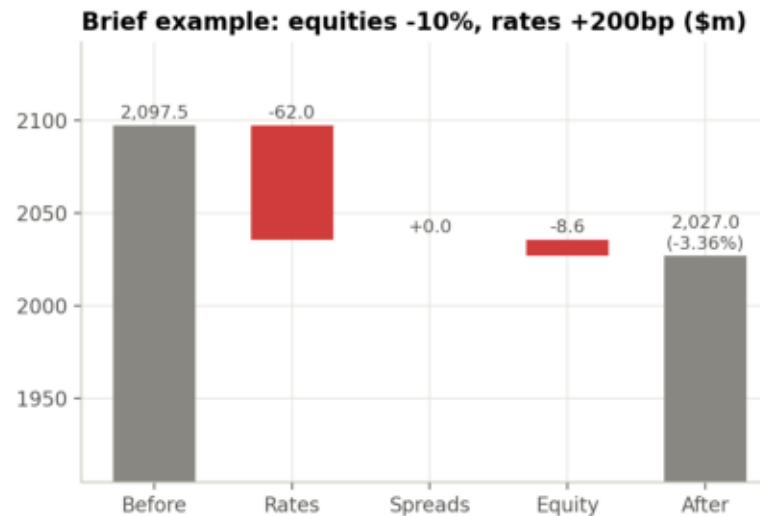
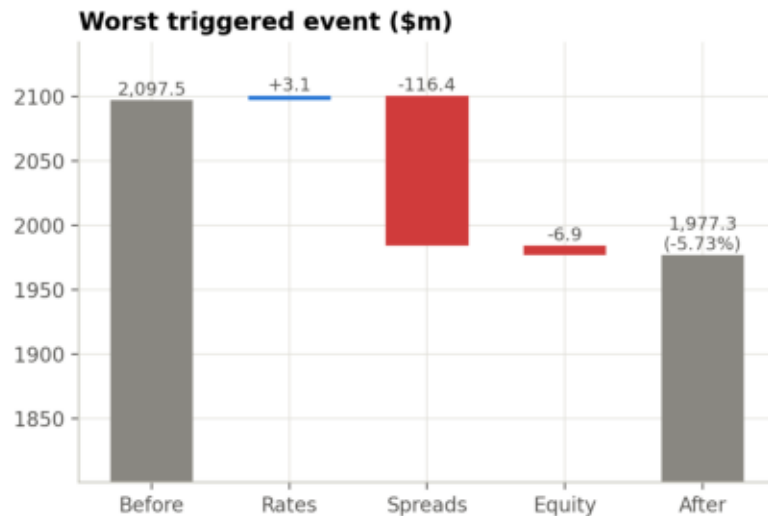
11.1%

avg one-way turnover / day

- Rule: decayed (36h) impact x source weighted sentiment $\rightarrow$ weight = $1/15 \times \exp(1.5 \times \text{score})$, bounded [2%, 15%], max 20% turnover/day; set at close t, earns t+1 (no look-ahead). Placebo with shuffled sentiment: mean -6.9% over 20 runs.

- Caveat: synthetic prices were generated from the same events as the text, so this outperformance is partly circular: it proves the plumbing works, not that the signal has alpha.

Results, Module B: event-triggered stress testing



- 369 signals with impact > 7 -> 248 distinct stress tests (24h cooldown per event+ticker; positive idiosyncratic news skipped).

- Worst: Credit Event on GS (impact 10): -5.73% = -\$120.2m on a \$2,098m book.

- Brief's example (-10% equity, +200bp): -3.36%; swaps and puts offset \$+77.8m.

- Valuing derivatives on market value instead of risk notional would show only \$+1.5m of hedge: the naive approach mis-states risk.

Event type	Triggers	Mean P&L	Worst P&L
Credit Event	54	-4.52%	-5.73%
Macroeconomic	23	-3.23%	-3.77%
Geopolitical	23	-2.61%	-3.17%
Regulatory/Legal	42	-0.58%	-0.81%
Earnings	48	-0.23%	-0.42%
Merger/Acquisition	20	-0.22%	-0.38%
Product Launch	38	-0.05%	-0.18%

Domain impact and challenges

Domain impact

- Portfolio managers: a transparent, rules-based tilt that reacts within one close to news flow, with turnover and concentration limits a risk committee can sign off.
- Risk / treasury desks: every high-impact headline becomes an immediate, explainable 'what does this do to our book' number, broken down by rates, credit and equity and by asset type and sector.
- One engine, many consumers: the pub/sub contract lets new modules (limits monitoring, alerts) subscribe without touching the NLP code.

Challenges

- No labelled real-time feed: built a synthetic generator plus 55 handwritten texts to measure generalisation honestly.
- Entity linking & leakage: masking company names so models read the language.
- Look-ahead: signals after 16:00 roll to the next rebalance; tested explicitly.
- Alert storms: 369 raw hits -> 248 triggers via strict threshold, cooldown and adverse-only rules.
- Derivatives: P&L on risk notional, not ~0 market value; sign tests for swaps, CDS, puts.

Limitations and next steps

Limitations (stated plainly)

- All data is synthetic. Text, prices and portfolio come from scripts/generate_data.py.
- Prices were generated from the same events as the text, so the Module A backtest is partly circular.
- Engine generalisation is measured on only 20 handwritten holdout texts.
- Impact learned template intensity words: 'War breaks out in the Middle East; oil spikes...' scores only 5.7.
- Stress model is first-order (duration / beta): no convexity, vega, correlations, rating migration or default loss.
- Scenario sizes are expert judgement, not calibrated to history.

Next steps

- Plug in real feeds (GDELT, NewsAPI, Kaggle stock tweets) behind the same ingest schema.
- Fine-tune a finance transformer (e.g. FinBERT) for sentiment and events; compare vs the lexicon baseline.
- Evaluate Module A on real prices (yfinance), with transaction costs and an event-study of signal decay.
- Calibrate shocks to historical episodes; add convexity, correlated factor shocks and credit migration.
- Replace the in-process bus with Kafka; stream signals to the dashboard live.